

Forecasting Pickup Densities For Yellow Taxi Services In New York City:



```
In [1]: #Use dask to read only necessary columns at a time. Speed up EDA
import dask.dataframe as dd#similar to pandas
import pandas as pd#pandas to create small dataframes
import folium#open street map
import datetime#Convert to unix time
import time#Convert to unix time
import numpy as np#Do arithmetic operations on arrays
import seaborn as sns#Plots
import matplotlib.pyplot as plt#Plots
import matplotlib #Plots
from matplotlib import rcParams#Size of plots
import gpxpy.geo#Get the haversine distance
from sklearn.cluster import MiniBatchKMeans, KMeans#Clustering
import math
import pickle
import os
mingw_path = 'C:\\Program Files\\mingw-w64\\x86_64-5.3.0-posix-seh-rt_v4-re
v0\\mingw64\\bin'
os.environ['PATH'] = mingw_path + ';' + os.environ['PATH']
import xgboost as xgb
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
```

```
In [2]: #There are 19 columns
month = dd.read_csv('yellow_tripdata_2015-01.csv')
print(month.columns)

Index(['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
      'passenger_count', 'trip_distance', 'pickup_longitude',
      'pickup_latitude', 'RateCodeID', 'store_and_fwd_flag',
      'dropoff_longitude', 'dropoff_latitude', 'payment_type', 'fare_amo
nt',
      'extra', 'mta_tax', 'tip_amount', 'tolls_amount',
      'improvement_surcharge', 'total_amount'],
      dtype='object')
```

Data Information

The yellow and green taxi trip records include fields capturing

- pick-up and drop-off dates/times,
- pick-up and drop-off locations,
- trip distances,
- itemized fares,
- rate types,
- payment types,
- driver-reported passenger counts.

The data used in the attached datasets were collected and provided to the NYC Taxi and Limousine Commission (TLC)

Information on taxis:

Yellow Taxi: Yellow Medallion Taxicabs

These are the famous NYC yellow taxis that provide transportation exclusively through street-hails. The number of taxicabs is limited by a finite number of medallions issued by the TLC. You access this mode of transportation by standing in the street and hailing an available taxi with your hand. The pickups are not pre-arranged.

For Hire Vehicles (FHVs)

FHV transportation is accessed by a pre-arrangement with a dispatcher or limo company. These FHVs are not permitted to pick up passengers via street hails, as those rides are not considered pre-arranged.

Green Taxi: Street Hail Livery (SHL)

The SHL program will allow livery vehicle owners to license and outfit their vehicles with green borough taxi branding, meters, credit card machines, and ultimately the right to accept street hails in addition to pre-arranged rides.

Credits: Quora

Footnote:

In the given notebook we are considering only the yellow taxis for the year of 2015

Data Collection

We Have collected all yellow taxi trips data from jan-2015 to dec-2016(Will be using only 2015 data)

file name	file name size	number of records	number of features
yellow_tripdata_2016-01	1. 59G	10906858	19
yellow_tripdata_2016-02	1. 66G	11382049	19
yellow_tripdata_2016-03	1. 78G	12210952	19
yellow_tripdata_2016-04	1. 74G	11934338	19
yellow_tripdata_2016-05	1. 73G	11836853	19
yellow_tripdata_2016-06	1. 62G	11135470	19
yellow_tripdata_2016-07	884Mb	10294080	17
yellow_tripdata_2016-08	854Mb	9942263	17
yellow_tripdata_2016-09	870Mb	10116018	17
yellow_tripdata_2016-10	933Mb	10854626	17
yellow_tripdata_2016-11	868Mb	10102128	17
yellow_tripdata_2016-12	897Mb	10449408	17
yellow_tripdata_2015-01	1.84Gb	12748986	19
yellow_tripdata_2015-02	1.81Gb	12450521	19
yellow_tripdata_2015-03	1.94Gb	13351609	19
yellow_tripdata_2015-04	1.90Gb	13071789	19
yellow_tripdata_2015-05	1.91Gb	13158262	19
yellow_tripdata_2015-06	1.79Gb	12324935	19
yellow_tripdata_2015-07	1.68Gb	11562783	19
yellow_tripdata_2015-08	1.62Gb	11130304	19
yellow_tripdata_2015-09	1.63Gb	11225063	19
yellow_tripdata_2015-10	1.79Gb	12315488	19
yellow_tripdata_2015-11	1.65Gb	11312676	19
yellow_tripdata_2015-12	1.67Gb	11460573	19

Features in the dataset:

Field Name	Description
VendorID	A code indicating the TPEP provider that provided the record. 1. Creative Mobile Technologies 2. VeriFone Inc.
tpcp_pickup_datetime	The date and time when the meter was engaged.
tpcp_dropoff_datetime	The date and time when the meter was disengaged.
Passenger_count	The number of passengers in the vehicle. This is a driver-entered value.
Trip_distance	The elapsed trip distance in miles reported by the taximeter.
Pickup_longitude	Longitude where the meter was engaged.
Pickup_latitude	Latitude where the meter was engaged.
RateCodeID	The final rate code in effect at the end of the trip. 1. Standard rate 2. JFK 3. Newark 4. Nassau or Westchester 5. Negotiated fare 6. Group ride
Store_and_fwd_flag	This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka "store and forward," because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip
Dropoff_longitude	Longitude where the meter was disengaged.
Dropoff_latitude	Latitude where the meter was disengaged.
Payment_type	A numeric code signifying how the passenger paid for the trip. 1. Credit card 2. Cash 3. No charge 4. Dispute 5. Unknown 6. Voided trip
Fare_amount	The time-and-distance fare calculated by the meter.
Extra	Miscellaneous extras and surcharges. Currently, this only includes. the 0.50 and 1 rush hour and overnight charges.
MTA_tax	0.50 MTA tax that is automatically triggered based on the metered rate in use.
Improvement_surcharge	0.30 improvement surcharge assessed trips at the flag drop. the improvement surcharge began being levied in 2015.
Tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
Tolls_amount	Total amount of all tolls paid in trip.

Total_amount	The total amount charged to passengers. Does not include cash tips.
--------------	---

```
In [3]: samples = 12748986+12450521+13351609+13071789+13158262+12324935+11562783+11
130304+11225063+12315488+11312676+11460573
print(samples)

146112989
```

Total number of samples for the year of 2015:

- 146 million

Problem statement(Change)

Find out the pick up density give the time and a region.

We will be taking the data of year 2015(12 months data) as train data, and test our model on year 2016 data.

EDA for the months JAN - DEC

Eventful days in january:

1. Thursday :January 01 New Years Day
2. Monday :January 19 Martin Luther King Jr. Day Third Monday in January
3. Sunday-Tuesday :January 25-27 Blizzard shuts down city

Eventful days in february:

1. Thursday :February 12 Lincoln's Birthday
2. Monday :February 16 Presidents' Day

Eventful days in March

1. No repeating events

```
In [4]: month.head(5)
```

```
Out[4]:
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_dis
0	2	2015-01-15 19:05:39	2015-01-15 19:23:42	1	1.59
1	1	2015-01-10 20:33:38	2015-01-10 20:53:28	1	3.30
2	1	2015-01-10 20:33:38	2015-01-10 20:43:41	1	1.80
3	1	2015-01-10 20:33:39	2015-01-10 20:35:31	1	0.50
4	1	2015-01-10 20:33:39	2015-01-10 20:52:58	1	3.00

Data Cleaning:

1. Removing the points outside of new york:

New york bounding box:

1. min_lat <- 40.5774
2. max_lat <- 40.9176
3. min_long <- -74.15
4. max_long <- -73.7004

```
In [5]: outlier_locations = month[(month.pickup_longitude != 0 and month.pickup_latitude != 0) & ((month.pickup_longitude <= -74.15) or (month.pickup_latitude <= 40.5774) or \
                                     (month.pickup_longitude >= -73.7004) or (month.pickup_latitude >= 40.9176))]
#Optional
#Comment out if necessary
map_osm = folium.Map(location=[40.734695, -73.990372], tiles='Stamen Toner')
sample_locations = outlier_locations.head(100)
for i,j in sample_locations.iterrows():
    folium.Marker(list((j['pickup_latitude'],j['pickup_longitude']))).add_to(map_osm)
map_osm
```

C:\Users\JackSparrow\AppData\Local\Continuum\Anaconda3\lib\site-packages\dataframe\core.py:3804: UserWarning: Insufficient elements for `head`. 100 elements requested, only 69 elements available. Try passing larger `npartitions` to `head`.

warnings.warn(msg.format(n, len(r)))

Out[5]:



Leaflet (<http://leafletjs.com>) | Map tiles by Stamen Design (<http://stamen.com>), under CC BY 3.0 (<http://creativecommons.org/licenses/by/3.0>). Data by OpenStreetMap (<http://openstreetmap.org>), under ODbL (<http://www.openstreetmap.org/copyright>).

As you can see above that there are some points just outside the boundary but many of them point to either South america, Mexico or Canada

2. Removing the outlier trips based on trip durations:

TLC Regulations:

- According to TLC regulations the maximum allowed trip duration in a 24 hour interval is 12 hours.
- This means the maximum allowed work hours for a driver is 12 hours
- There are 24k trips which either violate this rule or have trip durations of 0 or less than zero(Wierd)

```
In [6]: def convert_to_unix(s):
        return time.mktime(datetime.datetime.strptime(s, "%Y-%m-%d %H:%M:%S").timetuple())

#return a dataframe with many parameters along with trip duration and trip pickup times(with outliers)
def return_with_trip_times():
    duration = month[['tpep_pickup_datetime', 'tpep_dropoff_datetime']].compute()
    #pickups and dropoffs to unix time
    duration_pickup = [convert_to_unix(x) for x in duration['tpep_pickup_datetime'].values]
    duration_drop = [convert_to_unix(x) for x in duration['tpep_dropoff_datetime'].values]
    #calculate duration of trips
    durations = (np.array(duration_drop) - np.array(duration_pickup))/float(60)

    #append durations of trips and pickup times in unix to a new dataframe
    new_frame = month[['VendorID', 'payment_type', 'passenger_count', 'trip_distance', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'RateCodeID', 'store_and_fwd_flag']].compute()

    new_frame['trip_times'] = durations
    new_frame['pickup_times'] = duration_pickup
    new_frame['Speed'] = 60*(new_frame['trip_distance']/new_frame['trip_times'])

    return new_frame
#duration_dropoff

#outliers aren't still removed
#dataframe with durations and pickup unix time appended
frame_with_durations = return_with_trip_times()
```

```

In [7]: def plot_boxplot(new_frame, string1, string2):
        var = new_frame[string1].values
        # print(string1,var, new_frame.shape)
        var = np.sort(var,axis = None)
        # print(string1, var.shape)
        if string2!="":
            plt.ylim(0, var[int(float(95)*len(var)/100)])
            sns.boxplot(y= string1, data = new_frame)
            sns.plt.title(string1+' in minutes without outliers')
            plt.show()
        return var

def remove_outliers(new_frame):
    print ("Original:",new_frame.shape[0])
    a = new_frame.shape[0]
    new_frame = new_frame[((new_frame.dropoff_longitude >= -74.15) & (new_f
rame.dropoff_longitude <= -73.7004) & \
                            (new_frame.dropoff_latitude >= 40.5774) & (new_fram
e.dropoff_latitude <= 40.9176)) & \
                            ((new_frame.pickup_longitude >= -74.15) & (new_fram
e.pickup_latitude >= 40.5774)& \
                            (new_frame.pickup_longitude <= -73.7004) & (new_fram
e.pickup_latitude <= 40.9176))]
    print ("Removing outliers from NY boundaries:",new_frame.shape[0],(a -
new_frame.shape[0]))
    b = new_frame.shape[0]

    print ("\ntrip times percentiles...")
    var = plot_boxplot(new_frame, "trip_times", "")
    for i in range(0,len(var),int(float(10)*len(var)/100)):
        print ("{}th percentile {}".format(round(float(i)/len(var),3)*100,r
ound(var[i],3)))

    print(new_frame.shape, "before frist")
    # new_frame = new_frame[(new_frame.trip_times > 0) & (new_frame.trip_ti
mes < 720)]
    print(new_frame.shape, "frist", new_frame.head())
    print ("Removing outliers from trip times:",new_frame.shape[0],(b - new
_frame.shape[0]))
    var = plot_boxplot(new_frame, "trip_times", "out")
    print(new_frame.shape, "second")
    #-----
    print ("\ntrip distance percentiles...")
    var = plot_boxplot(new_frame, "trip_distance", "")
    for i in range(0,len(var),int(float(10)*len(var)/100)):
        print ("{}th percentile {}".format(round(float(i)/len(var),3)*100,r
ound(var[i],3)))

    c = new_frame.shape[0]
    new_frame = new_frame[(new_frame.trip_distance > 0) & (new_frame.trip_d
istance < 23)]
    print ("Removing outliers from trip distance:",new_frame.shape[0],(c -
new_frame.shape[0]))
    var = plot_boxplot(new_frame,"trip_distance", "out")
    #-----

    print ("\nSpeed percentiles...")
    var = plot_boxplot(new_frame, "Speed", "")
    for i in range(0,len(var),int(float(10)*len(var)/100)):

```



```
        print ("{}th percentile {}".format(round(float(i)/len(var),3)*100,round(var[i],3)))

    d = new_frame.shape[0]
    new_frame = new_frame[(new_frame.Speed <= 65) & (new_frame.Speed >= 0)]
    print ("Removing outliers from trip speed:",new_frame.shape[0],(d - new_frame.shape[0]))
    var = plot_boxplot(new_frame, "Speed", "out")

#-----
    print( "Total outliers removed",a - new_frame.shape[0])
    return new_frame

frame_with_durations_outliers_removed = remove_outliers(frame_with_durations)
```

Original: 12748986

Removing outliers from NY boundaries: 12455067 293919

trip times percentiles...

0.0th percentile -1211.017

10.0th percentile 3.867

20.0th percentile 5.4

30.0th percentile 6.833

40.0th percentile 8.317

50.0th percentile 9.95

60.0th percentile 11.867

70.0th percentile 14.267

80.0th percentile 17.583

90.0th percentile 23.333

100.0th percentile 23800.017

(12455067, 13) before frist

(12455067, 13) frist	VendorID	payment_type	passenger_count	trip_distance	pickup_longitude
0	2	1	1.59	-73.99389	6
1	1	1	3.30	-74.00164	8
2	1	2	1.80	-73.96334	1
3	1	2	0.50	-74.00908	7
4	1	2	3.00	-73.97117	6

	pickup_latitude	dropoff_longitude	dropoff_latitude	RateCodeID	\
0	40.750111	-73.974785	40.750618	1	
1	40.724243	-73.994415	40.759109	1	
2	40.802788	-73.951820	40.824413	1	
3	40.713818	-74.004326	40.719986	1	
4	40.762428	-74.004181	40.742653	1	

	store_and_fwd_flag	trip_times	pickup_times	Speed
0	N	18.050000	1.421329e+09	5.285319
1	N	19.833333	1.420902e+09	9.983193
2	N	10.050000	1.420902e+09	10.746269
3	N	1.866667	1.420902e+09	16.071429
4	N	19.316667	1.420902e+09	9.318378

Removing outliers from trip times: 12455067 0



(12455067, 13) second

trip distance percentiles...

0.0th percentile 0.0

10.0th percentile 0.65

20.0th percentile 0.9

30.0th percentile 1.1

40.0th percentile 1.37

50.0th percentile 1.68

60.0th percentile 2.06

70.0th percentile 2.6

80.0th percentile 3.57

90.0th percentile 5.9

100.0th percentile 800016.5

Removing outliers from trip distance: 12399797 55270



Speed percentiles...

0.0th percentile -0.228

10.0th percentile 6.413

20.0th percentile 7.811

30.0th percentile 8.929

40.0th percentile 9.98

50.0th percentile 11.066

60.0th percentile 12.281

70.0th percentile 13.785

80.0th percentile 15.939

90.0th percentile 20.104

100.0th percentile inf

Removing outliers from trip speed: 12391863 7934



Total outliers removed 357123

In [8]: `frame_with_durations_outliers_removed.head()`

Out[8]:

	VendorID	payment_type	passenger_count	trip_distance	pickup_longitude	pickup
0	2	1	1	1.59	-73.993896	40.750
1	1	1	1	3.30	-74.001648	40.724
2	1	2	1	1.80	-73.963341	40.802
3	1	2	1	0.50	-74.009087	40.713
4	1	2	1	3.00	-73.971176	40.762

```
In [9]: import seaborn as sns
#trip times
def dist_of_params(frame,variable,title):
    sns.FacetGrid(frame,size=6) \
        .map(sns.kdeplot, variable) \
        .add_legend();
    sns.plt.title(title)
    plt.show();

#trip times
dist_of_params(frame_with_durations_outliers_removed,'trip_times','time for
cab trips distribution(in minutes)')

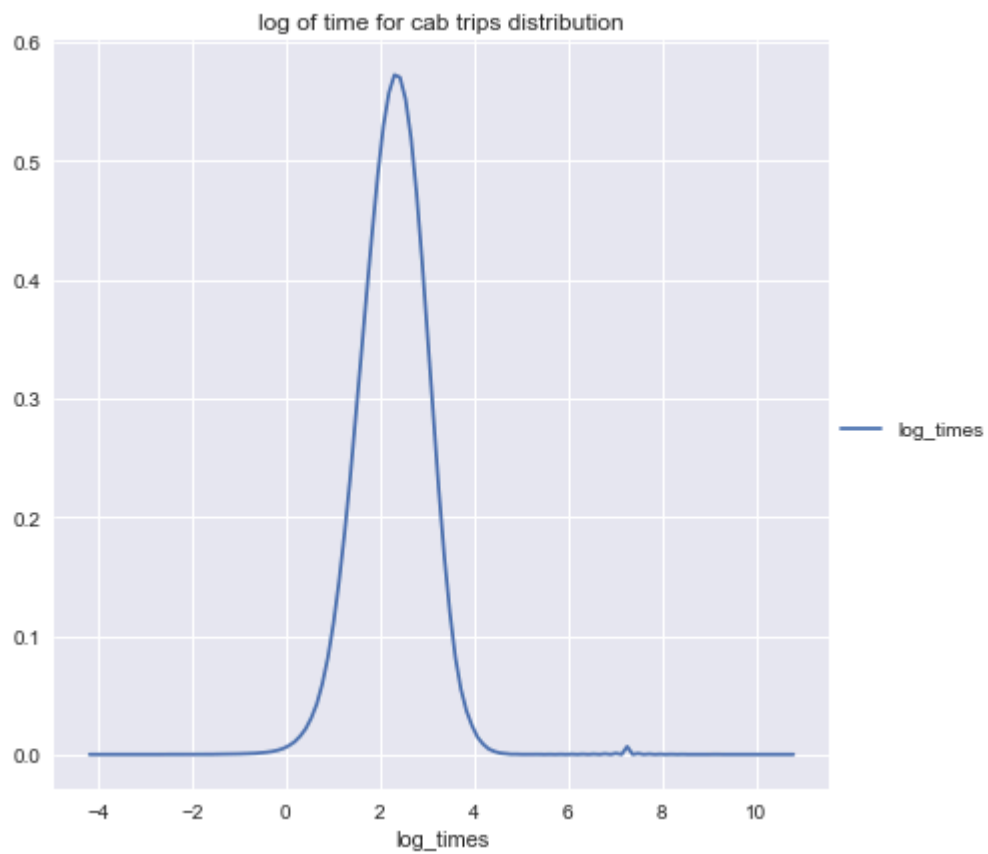
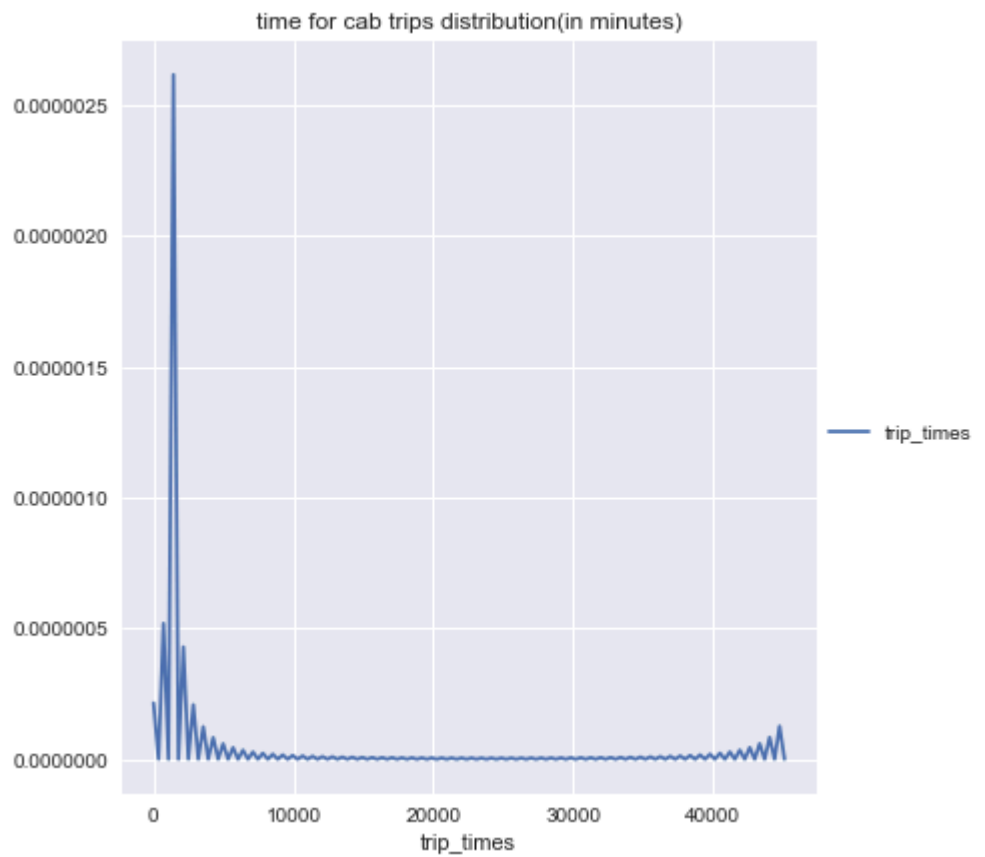
#Log trip times
log_trip_times = frame_with_durations_outliers_removed.trip_times.values
frame_with_durations_outliers_removed['log_times'] = np.log(log_trip_times)
dist_of_params(frame_with_durations_outliers_removed,'log_times','log of ti
me for cab trips distribution')

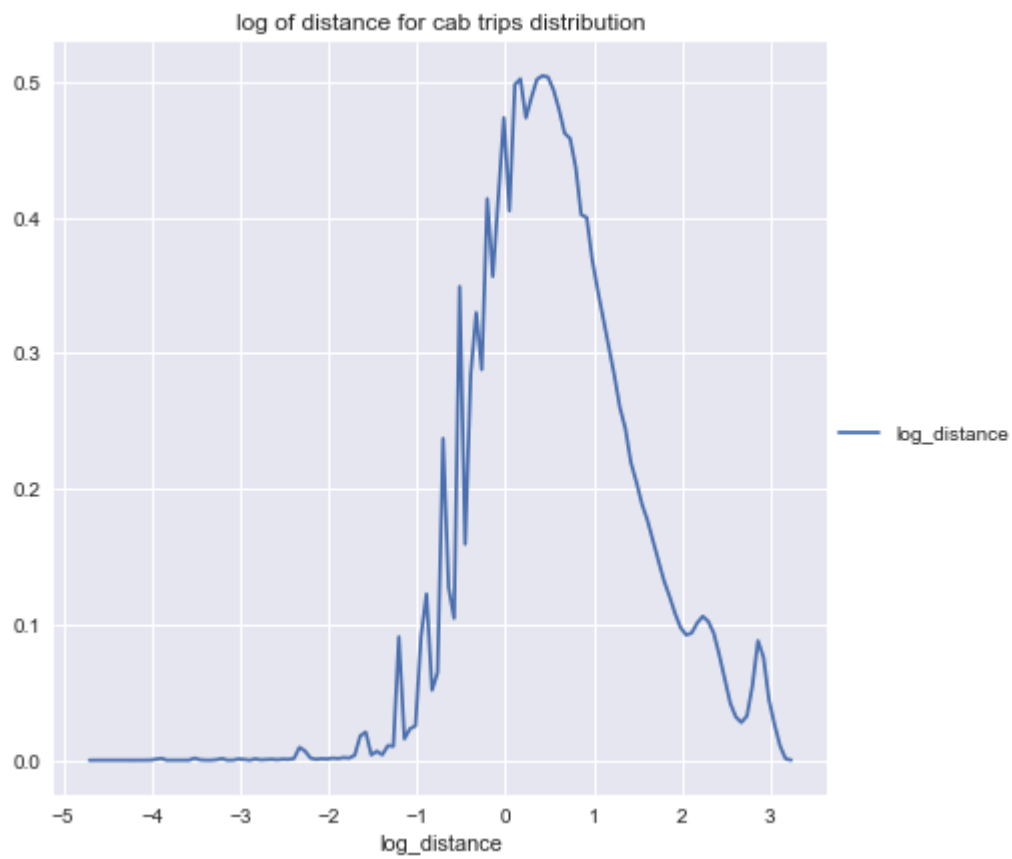
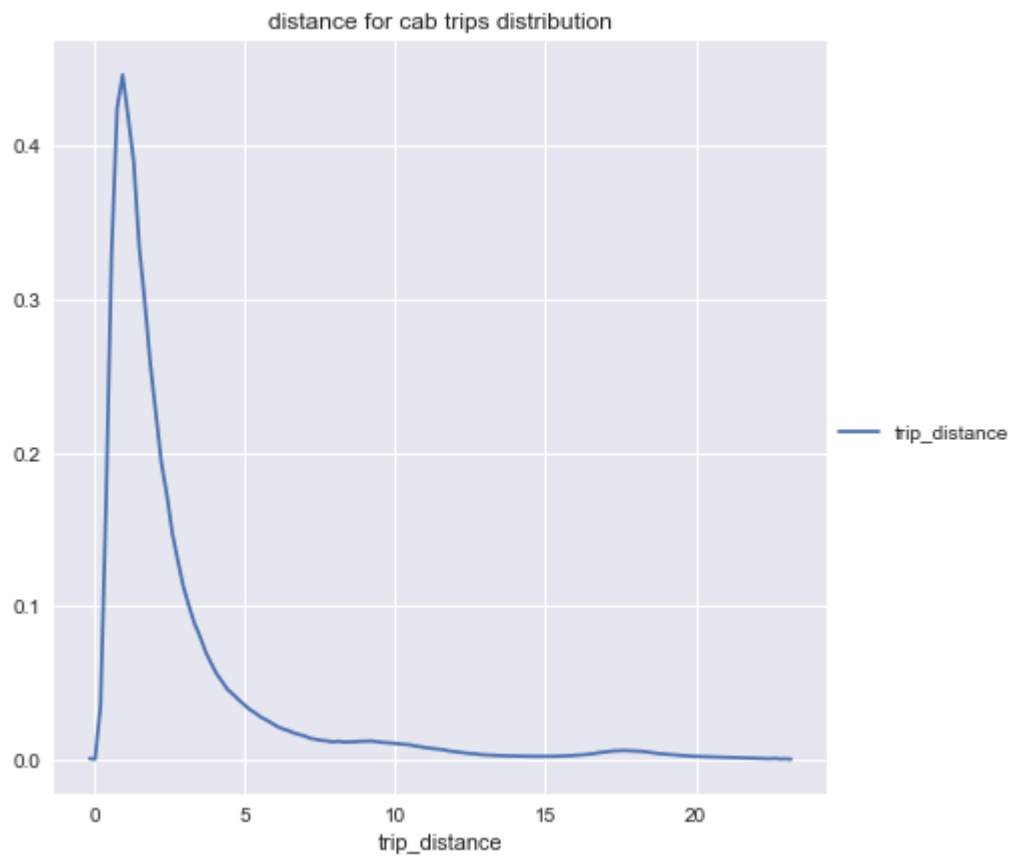
#trip distances
dist_of_params(frame_with_durations_outliers_removed,'trip_distance','dista
nce for cab trips distribution')

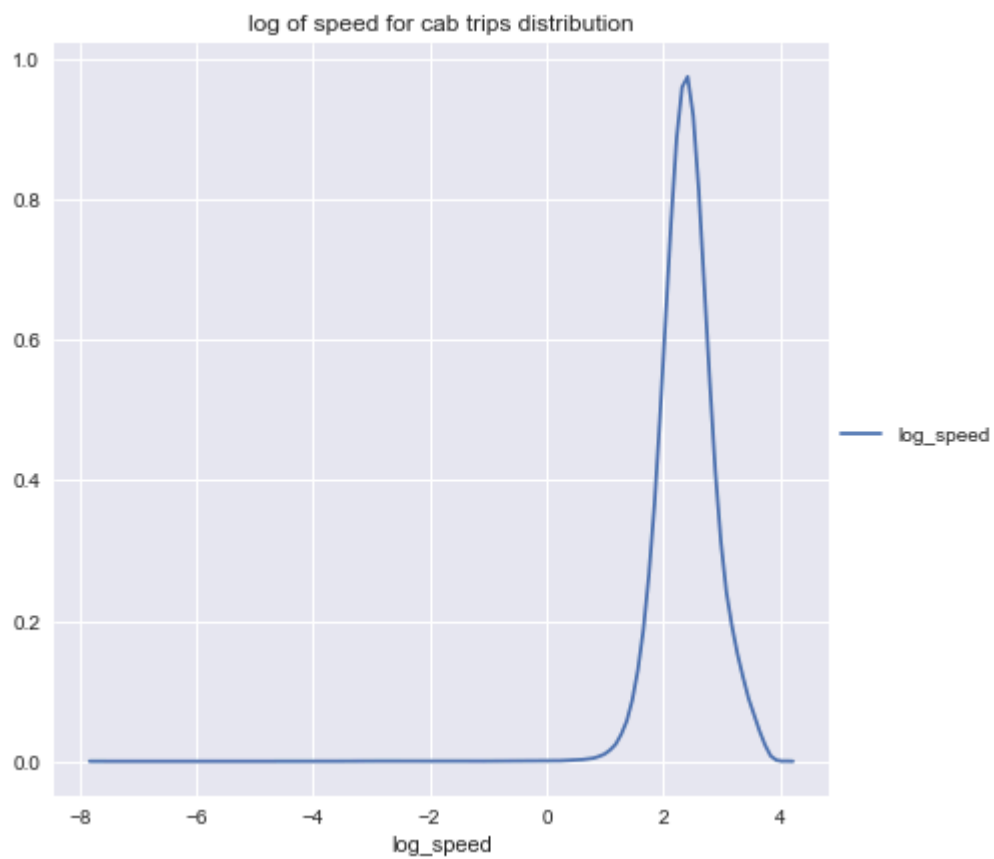
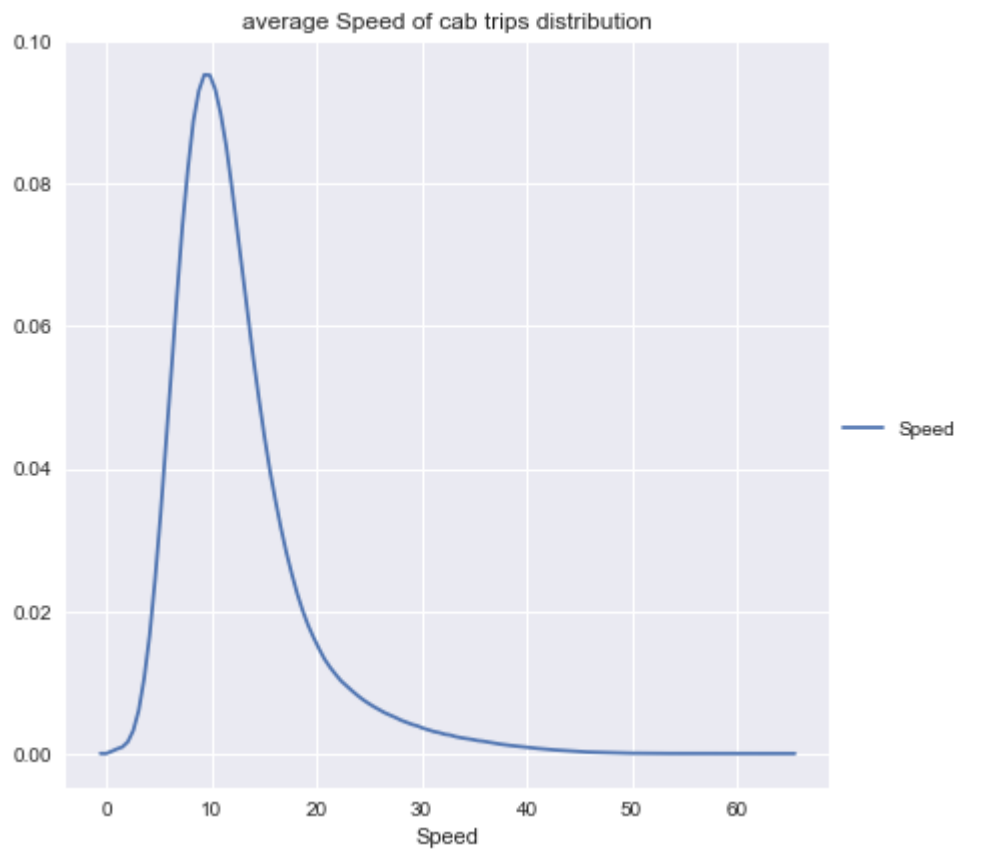
#Log trip distances
log_trip_distance = frame_with_durations_outliers_removed.trip_distance.val
ues
frame_with_durations_outliers_removed['log_distance'] = np.log(log_trip_dis
tance)
dist_of_params(frame_with_durations_outliers_removed,'log_distance','log of
distance for cab trips distribution')

#trip speed
dist_of_params(frame_with_durations_outliers_removed,'Speed','average Speed
of cab trips distribution')

#Log speed
log_trip_speed = frame_with_durations_outliers_removed.Speed.values
frame_with_durations_outliers_removed['log_speed'] = np.log(log_trip_speed)
dist_of_params(frame_with_durations_outliers_removed,'log_speed','log of sp
eed for cab trips distribution')
```







Univariate Analysis of following features :

1. Vendor ID
2. Passenger counts
3. Rate Code ID
4. store and forward flags
5. Payment types

Outlier Trip Distances:

Outlier Dropoff and pickup points:

To simplify things new york has been divided into 37km x 37km box and any pickups or dropoffs outside this point is referred to as an outlier

1. There are 264.4k dropoffs happening outside of new york
2. There are 247.742 pickups happening outside of new york
3. How many travels have both drop off and pick ups outside of new york??
 - sdfsd

Inspecting number of pickups shared among each of the vendors

```
In [10]: #Get unique vendors
vendor_unique = frame_with_durations_outliers_removed.VendorID.unique()
Vendors = []
#Get the number of trips for each vendor
Vendors.append(frame_with_durations_outliers_removed[frame_with_durations_o
utliers_removed['VendorID'] == 1].shape[0])
Vendors.append(frame_with_durations_outliers_removed[frame_with_durations_o
utliers_removed['VendorID'] == 2].shape[0])
#Names of vendors
print("Unique vendors\n {}: Creative Mobile Technologies\n {}: VeriFone Inc
\n".format(vendor_unique[1],vendor_unique[0]))
print ("Vendor 1 and Vendor 2 count:\n",Vendors)
```

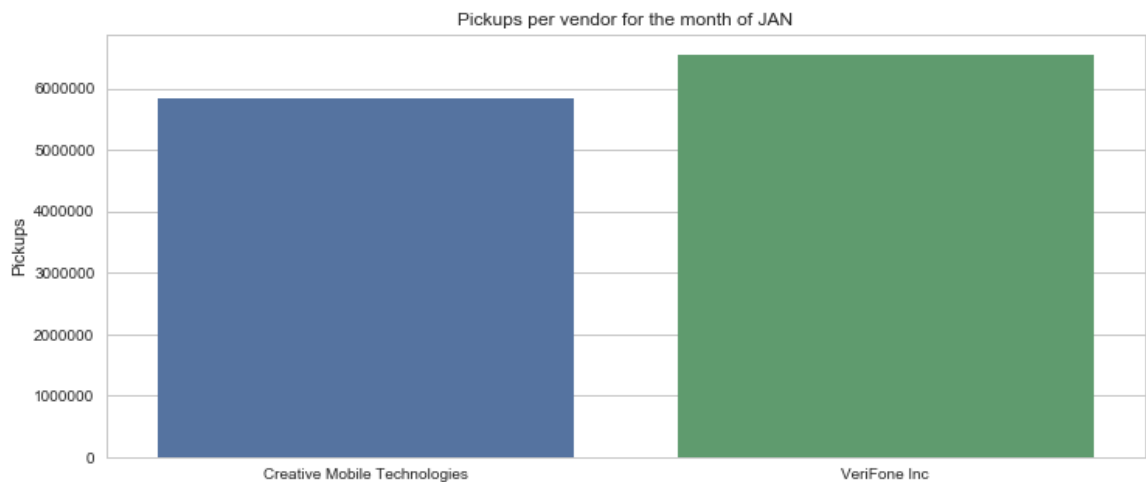
Unique vendors
 1: Creative Mobile Technologies
 2: VeriFone Inc

Vendor 1 and Vendor 2 count:
 [5841962, 6549901]

```
In [11]: sns.set_style("whitegrid")

fig, ax = plt.subplots()
fig.set_size_inches(12, 5)

ax = sns.barplot(x = ['Creative Mobile Technologies', 'VeriFone Inc'] , y =
np.array(Vendors))
ax.set(ylabel='Pickups')
sns.plt.title('Pickups per vendor for the month of JAN')
plt.show()
```



```
In [12]: print ("Creative Mobile Technologies share :{} % VeriFone Inc share :{} %
".format(100*float(Vendors[1])/sum(Vendors),100*float(Vendors[0])/sum(Vendors)))
```

Creative Mobile Technologies share :52.85646718334443 % VeriFone Inc share :47.14353281665557 %

Both the vendors share almost same number of pickups in JAN month

```
In [13]: #Get the number of pickups per each passenger count
passenger_count = []
passenger_uniq = set(frame_with_durations_outliers_removed['passenger_count'].values)
for i in passenger_uniq:
    passenger_count.append(frame_with_durations_outliers_removed[frame_with_durations_outliers_removed['passenger_count'] == i].shape[0])

#sort the passenger pickups based on the passenger count
pass_count_pickups = [x for _,x in sorted(zip(passenger_uniq,passenger_count))]
pass_count = [int(y) for y,_ in sorted(zip(passenger_uniq,passenger_count))]

print (pass_count)
print (pass_count_pickups)
```

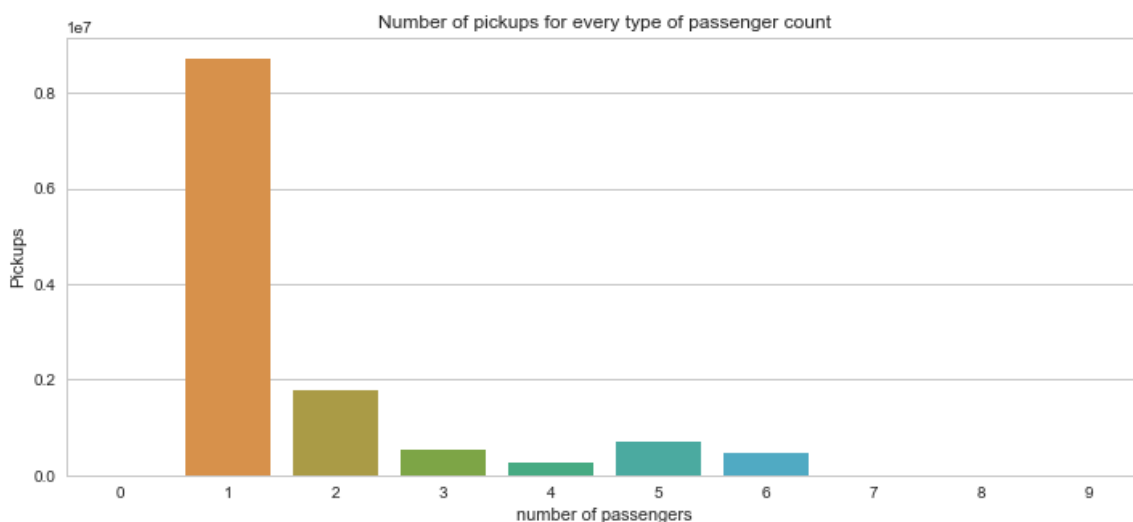
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[5841, 8717237, 1767371, 516166, 247081, 689157, 448998, 6, 2, 4]

```
In [14]: sns.set_style("whitegrid")

fig, ax = plt.subplots()
fig.set_size_inches(12, 5)

ax = sns.barplot(x = np.array(pass_count) , y = np.array(pass_count_pickups))
ax.set(ylabel='Pickups',xlabel = 'number of passengers')
sns.plt.title('Number of pickups for every type of passenger count')
plt.show()
```



```
In [15]: for a,b in zip(pass_count,pass_count_pickups):
          print ("Pickups for passenger count {} : {}% of total pickups".format(int(a),round(float(b)*100/sum(pass_count_pickups),4)))
```

```
Pickups for passenger count 0 : 0.0471% of total pickups
Pickups for passenger count 1 : 70.3465% of total pickups
Pickups for passenger count 2 : 14.2624% of total pickups
Pickups for passenger count 3 : 4.1654% of total pickups
Pickups for passenger count 4 : 1.9939% of total pickups
Pickups for passenger count 5 : 5.5614% of total pickups
Pickups for passenger count 6 : 3.6233% of total pickups
Pickups for passenger count 7 : 0.0% of total pickups
Pickups for passenger count 8 : 0.0% of total pickups
Pickups for passenger count 9 : 0.0% of total pickups
```

Inference from the passenger count:

1. Majority of the trips seem to have 1 as the ideal number of passengers with 2 as the close second
2. What do the passenger count with 0 say?
 - Human error? -
 - Maybe some form of goods delivery?
3. According to the TLC regulations 5 is the maximum number of passengers allowed in a yellow taxi
 - Good amount of trips have more than 5 passengers - almost 45k of them
 - Toddlers and children below age 7 are allowed to sit on laps
 - Maybe the drivers take into account the toddlers too(But still 7 ,8 and 9 are ridiculous numbers for passenger counts)

Rate Code:

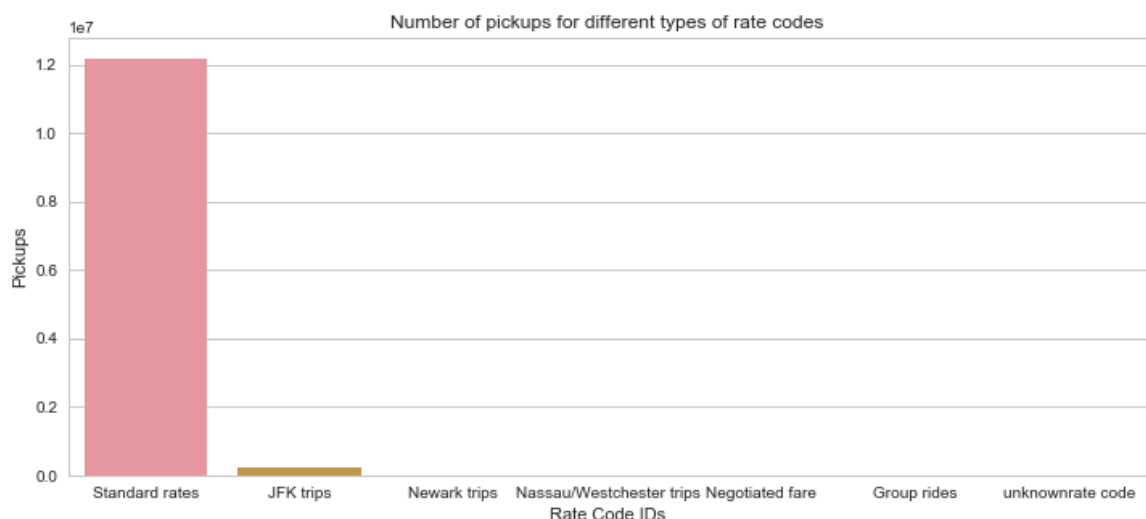
1. Standard rate
2. JFK
3. Newark
4. Nassau or Westchester
5. Negotiated fare
6. Group ride

```
In [16]: #Unique rate codes
rate_codes_uniq = set(frame_with_durations_outliers_removed['RateCodeID'].values)
rate_count = []
for i in rate_codes_uniq:
    rate_count.append(frame_with_durations_outliers_removed[frame_with_durations_outliers_removed['RateCodeID'] == i].shape[0])
#sort the pickups based on the rate codes
rate_code_pickups = [x for _, x in sorted(zip(rate_codes_uniq, rate_count))]
rate_code = [int(y) for y, _ in sorted(zip(rate_codes_uniq, rate_count))]
Trips = ['Standard rates', 'JFK trips', 'Newark trips', 'Nassau/Westchester trips', 'Negotiated fare', 'Group rides', 'unknownrate code']
```

```
In [17]: sns.set_style("whitegrid")

fig, ax = plt.subplots()
fig.set_size_inches(12, 5)

ax = sns.barplot(x = np.array(Trips) , y = np.array(rate_code_pickups))
ax.set(ylabel='Pickups', xlabel = 'Rate Code IDs')
sns.plt.title('Number of pickups for different types of rate codes')
plt.show()
```



```
In [18]: for a,b in zip(Trips,rate_code_pickups):
          print ("Pickups for/with {} accounts {}% of total pickups".format(a,round(float(b)*100/sum(rate_code_pickups),4)))
```

Pickups for/with Standard rates accounts 98.2089% of total pickups
 Pickups for/with JFK trips accounts 1.6787% of total pickups
 Pickups for/with Newark trips accounts 0.0072% of total pickups
 Pickups for/with Nassau/Westchester trips accounts 0.0133% of total pickups
 Pickups for/with Negotiated fare accounts 0.0894% of total pickups
 Pickups for/with Group rides accounts 0.0003% of total pickups
 Pickups for/with unknownrate code accounts 0.0023% of total pickups

Rate Codes(Findings from the data):

1. Standard rate - 12.4 Million trips
2. JFK - 224.2k trips
3. Newark - 17.7k trips
4. Nassau or Westchester - 4.1k trips
5. Negotiated fare - 36.8k trips
6. Group ride - 134 trips
7. rate code 99(mistake??) - 507 trips

Store and forward flags

```
In [19]: SWflag_uniq = set(frame_with_durations_outliers_removed['store_and_fwd_flag'].values)
          SWflag_count = []
          for i in SWflag_uniq :
              SWflag_count.append(frame_with_durations_outliers_removed[frame_with_durations_outliers_removed['store_and_fwd_flag'] == i].shape[0])
          print (SWflag_count)
          SWflag_uniq = list(SWflag_uniq)
          print (SWflag_uniq)
```

```
[12286027, 105836]
['N', 'Y']
```

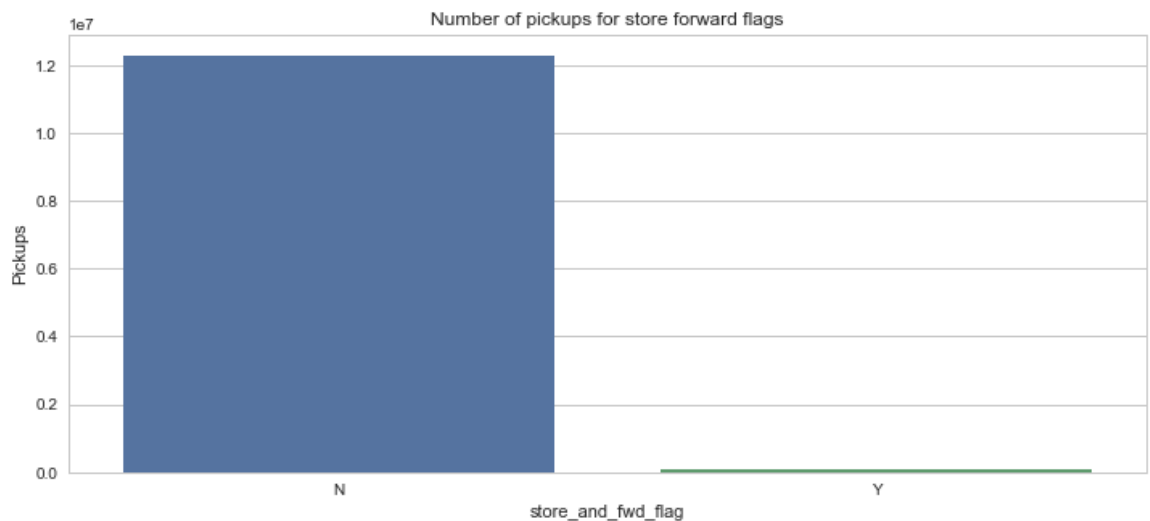
```
In [20]: for a,b in zip(SWflag_uniq,SWflag_count):
          print ("Pickups for/with store and forward = {} accounts {}% of total pickups".format(a,round(float(b)*100/sum(SWflag_count),4)))
```

Pickups for/with store and forward = N accounts 99.1459% of total pickups
 Pickups for/with store and forward = Y accounts 0.8541% of total pickups

```
In [21]: sns.set_style("whitegrid")

fig, ax = plt.subplots()
fig.set_size_inches(12, 5)

ax = sns.barplot(x = np.array(SWflag_uniq) , y = np.array(SWflag_count))
ax.set(ylabel='Pickups',xlabel = 'store_and_fwd_flag')
sns.plt.title('Number of pickups for store forward flags')
plt.show()
```



Payment types:

1. Credit card
2. Cash
3. No charge
4. Dispute
5. Unknown
6. Voided trip

```
In [22]: payment_uniq = set(frame_with_durations_outliers_removed['payment_type'].values)
payment_count = []
for i in payment_uniq :
    payment_count.append(frame_with_durations_outliers_removed[frame_with_durations_outliers_removed['payment_type'] == i].shape[0])
print (payment_count)
payment_uniq = list(payment_uniq)
print (payment_uniq)
```

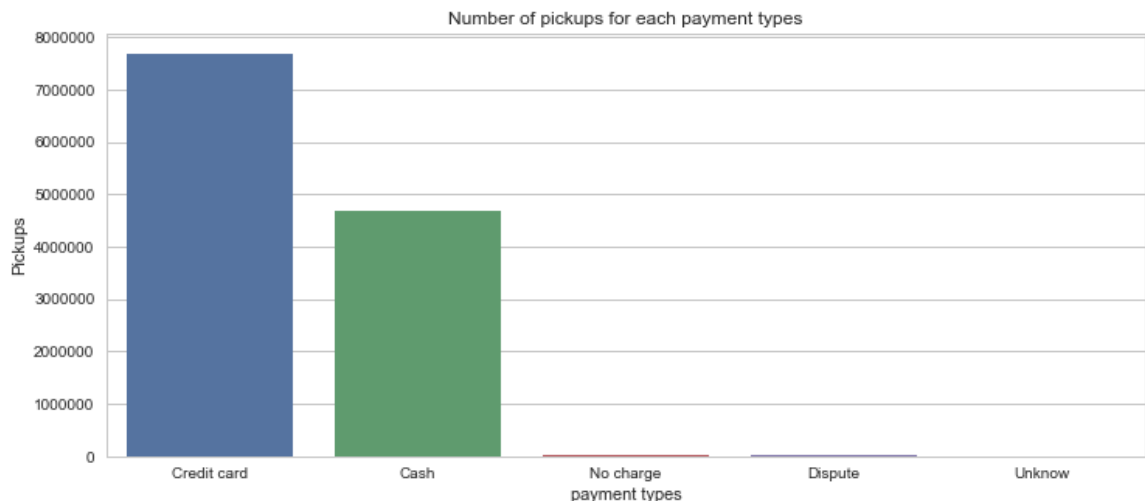
```
[7685155, 4670834, 26634, 9238, 2]
[1, 2, 3, 4, 5]
```

```
In [23]: pay_pickups = [x for _,x in sorted(zip(payment_uniq,payment_count))]
pay = [int(y) for y,_ in sorted(zip(payment_uniq,payment_count))]
pay_type = ['Credit card','Cash','No charge','Dispute','Unknow']
```

```
In [24]: sns.set_style("whitegrid")

fig, ax = plt.subplots()
fig.set_size_inches(12, 5)

ax = sns.barplot(x = np.array(pay_type) , y = np.array(pay_pickups))
ax.set(ylabel='Pickups',xlabel = 'payment types')
sns.plt.title('Number of pickups for each payment types')
plt.show()
```



```
In [25]: for a,b in zip(pay_type,pay_pickups):
          print ("Pickups with payment types {} accounts to :{}% of total pickup
s".format(a,round(float(b)*100/sum(pay_pickups),4)))
```

Pickups with payment types Credit card accounts to :62.0178% of total pickups
 Pickups with payment types Cash accounts to :37.6928% of total pickups
 Pickups with payment types No charge accounts to :0.2149% of total pickups
 Pickups with payment types Dispute accounts to :0.0745% of total pickups
 Pickups with payment types Unknow accounts to :0.0% of total pickups

Pickup and dropoff locations on maps:

```
In [26]: frame_with_durations_outliers_removed['lat_bin'] = np.round(frame_with_dura
tions_outliers_removed['pickup_latitude'],4)
frame_with_durations_outliers_removed['lon_bin'] = np.round(frame_with_dura
tions_outliers_removed['pickup_longitude'],4)

coords = frame_with_durations_outliers_removed[['pickup_latitude', 'pickup_
longitude']].values

#Getting 40 clusters using the kmeans
kmeans = MiniBatchKMeans(n_clusters=40, batch_size=10000).fit(coords)
frame_with_durations_outliers_removed['pickup_cluster'] = kmeans.predict(fr
ame_with_durations_outliers_removed[['pickup_latitude', 'pickup_longitud
e']])
frame_with_durations_outliers_removed['dropoff_cluster'] = kmeans.predict(f
rame_with_durations_outliers_removed[['dropoff_latitude', 'dropoff_longitud
e']])
```

```

In [27]: #Get regions
def Convert_Clusters(frame,cluster):
    region=[]
    colors = []
    Queens=[4,16,22,23,26,35,36]
    Brooklyn=[11,19,29,37]
    for i in frame[cluster].values:
        if i==2:
            region.append("JFK")
            #Green - JFK
            colors.append('#7CFC00')
        elif i==13:
            region.append("Bronx")
            #Red - Bronx
            colors.append('#DC143C')
        elif i in Queens:
            region.append("Queens")
            #Blue - Queens
            colors.append('#00FFFF')
        elif i in Brooklyn:
            region.append("Brooklyn")
            #Brooklyn - yellow orange
            colors.append('#FFD700')
        else:
            region.append("Manhattan")
            #White - manhattan
            colors.append('#FFFFFF')
    frame['Regions '+cluster]=region
    return frame,colors

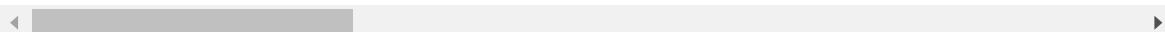
frame_with_durations_outliers_removed,colors_pickup=Convert_Clusters(frame_
with_durations_outliers_removed,'pickup_cluster')
frame_with_durations_outliers_removed,colors_dropoff=Convert_Clusters(frame
_with_durations_outliers_removed,'dropoff_cluster')
frame_with_durations_outliers_removed.head()

```

Out[27]:

	VendorID	payment_type	passenger_count	trip_distance	pickup_longitude	pickup
0	2	1	1	1.59	-73.993896	40.750
1	1	1	1	3.30	-74.001648	40.724
2	1	2	1	1.80	-73.963341	40.802
3	1	2	1	0.50	-74.009087	40.713
4	1	2	1	3.00	-73.971176	40.762

5 rows × 22 columns




```
In [28]: def plot_scatter_locations(frame,colors,choose):
          %matplotlib inline

          pd.options.display.mpl_style = 'default' #Better Styling
          new_style = {'grid': False} #Remove grid
          matplotlib.rc('axes', **new_style)
          rcParams['figure.figsize'] = (17.5, 17) #Size of figure
          rcParams['figure.dpi'] = 250

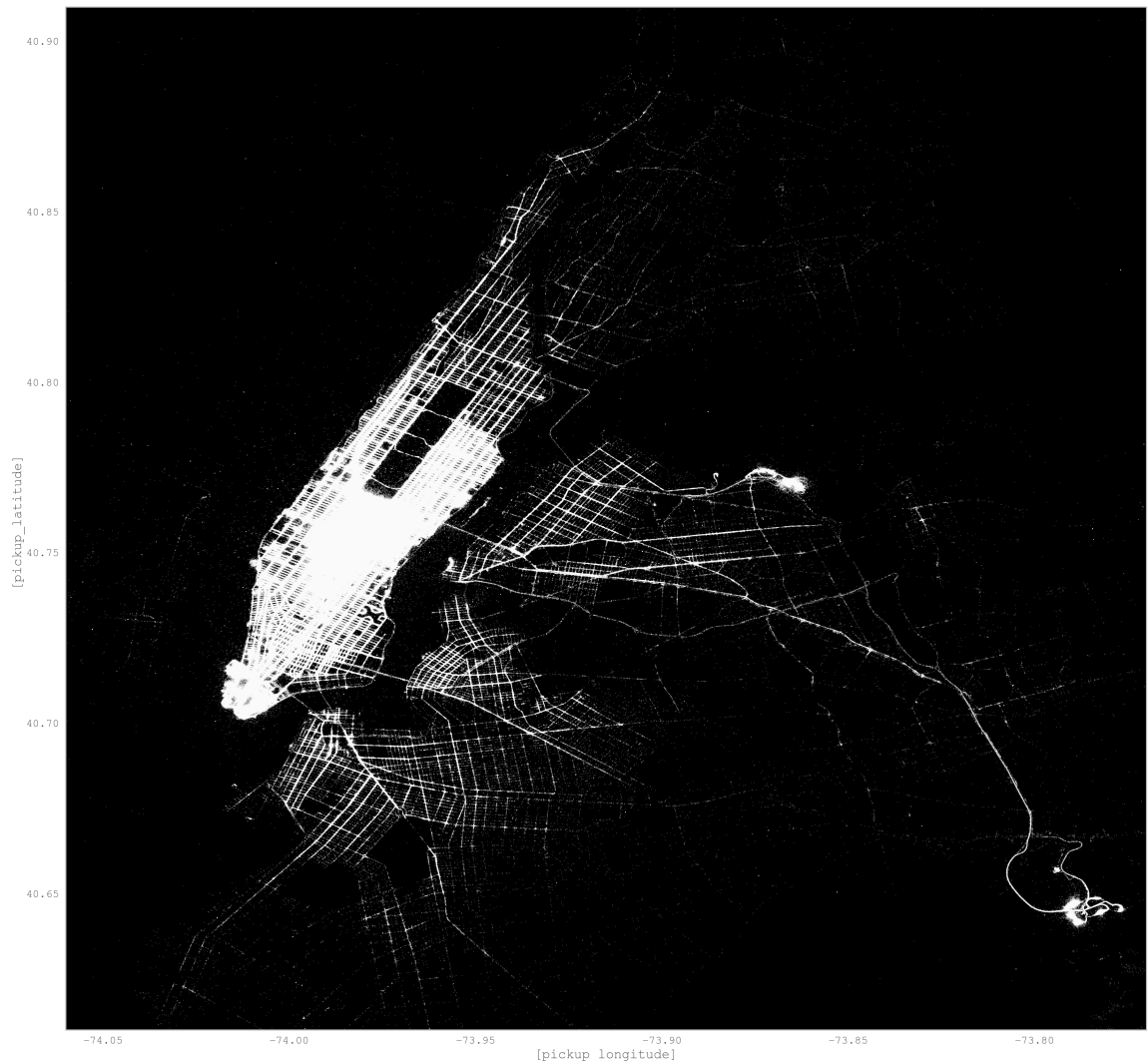
          var1 = choose+'_longitude'
          var2 = choose+'_latitude'

          P=frame.plot(kind='scatter', x=[var1], y=[var2],color='white',xlim=(-7
4.06,-73.77),ylim=(40.61, 40.91),s=.02,alpha=.6)
          P.set_axis_bgcolor('black') #Background Color
          plt.savefig('plot_'+choose+'.png')
          plt.show()

          plot_scatter_locations(frame_with_durations_outliers_removed,colors_picku
p,'pickup')
          plot_scatter_locations(frame_with_durations_outliers_removed,colors_dropof
f,'dropoff')
```

```
C:\Users\JackSparrow\AppData\Local\Continuum\Anaconda3\lib\site-packages\ipykernel_launcher.py:19: FutureWarning:
mpl_style had been deprecated and will be removed in a future version.
Use `matplotlib.pyplot.style.use` instead.
```

```
C:\Users\JackSparrow\AppData\Local\Continuum\Anaconda3\lib\site-packages\ipykernel_launcher.py:14: MatplotlibDeprecationWarning: The set_axis_bgcolor function was deprecated in version 2.0. Use set_facecolor instead.
```



```
C:\Users\JackSparrow\AppData\Local\Continuum\Anaconda3\lib\site-packages\ipykernel_launcher.py:20: FutureWarning:
mpl_style had been deprecated and will be removed in a future version.
Use `matplotlib.pyplot.style.use` instead.
```

**INFERENCE:**

1. The pickup locations seem to be concentrated in the manhattan area
2. The dropoff locations seem to be spread over the brooklyn and queens as well along with the manhattan area

```

In [29]: def compute_pickups_per_day():
    time = month[['tpep_pickup_datetime', 'tpep_dropoff_datetime']].compute
    ()
    time['tpep_pickup_datetime'] = pd.to_datetime(time['tpep_pickup_datetim
    e'])
    time["pickup_day"] = time['tpep_pickup_datetime'].dt.strftime('%u').ast
    ype(int)
    time["pickup_hour"] = time['tpep_pickup_datetime'].dt.strftime('%H').as
    type(int)
    set(time['pickup_day'].values)
    day_pickups = []
    night_pickups = []
    mid_night_pickups = []
    afternoon_pickups = []
    for i in range(1,8):
        day_pickups.append(time[(time.pickup_day == i) & (time.pickup_hour
        >= 6) & (time.pickup_hour < 12)].shape[0])
        night_pickups.append(time[(time.pickup_day == i) & (time.pickup_hou
        r >= 18) & (time.pickup_hour <= 23)].shape[0])
        mid_night_pickups.append(time[(time.pickup_day == i) & (time.pickup
        _hour >= 0) & (time.pickup_hour < 6)].shape[0])
        afternoon_pickups.append(time[(time.pickup_day == i) & (time.pickup
        _hour >= 12) & (time.pickup_hour < 18)].shape[0])

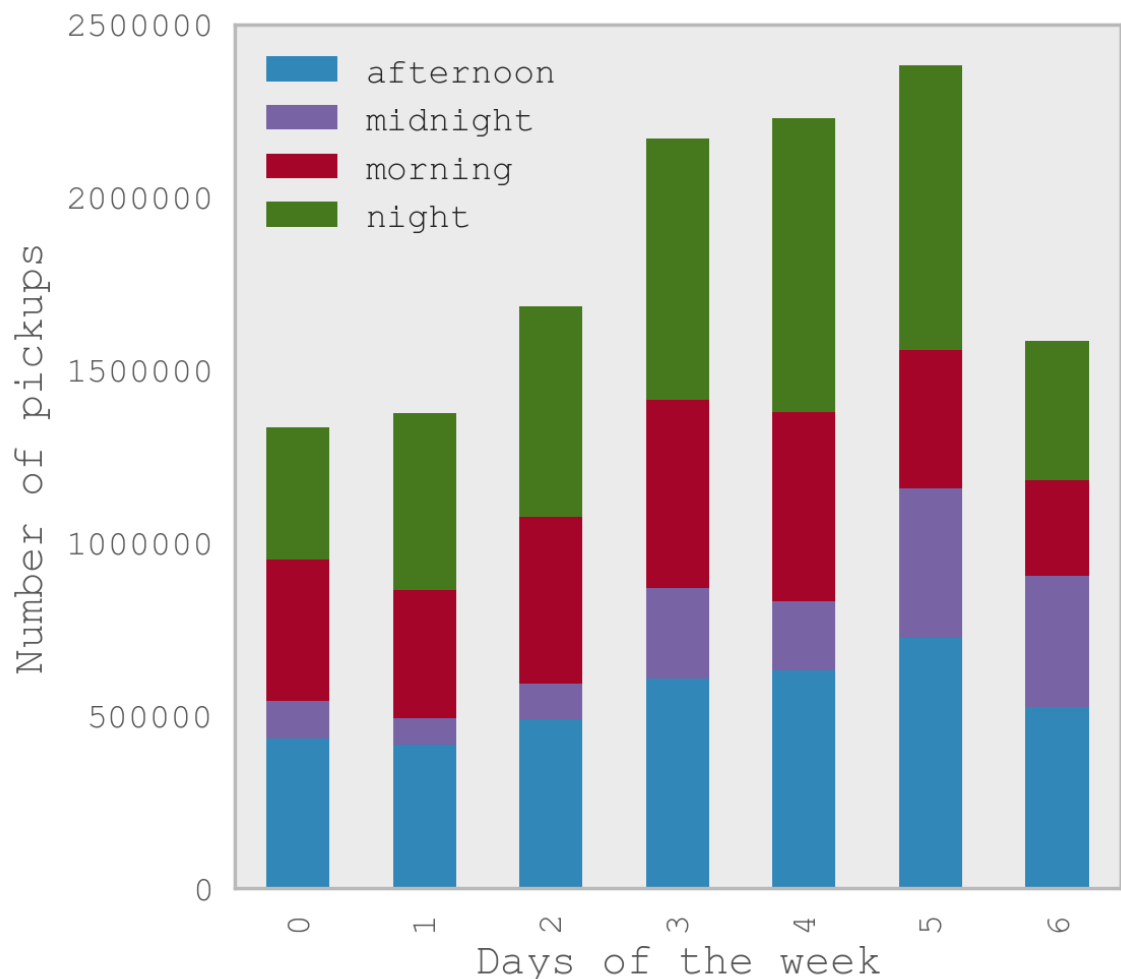
    days =pd.DataFrame({'morning':day_pickups, 'afternoon':afternoon_pickup
    s, 'night':night_pickups, 'midnight':mid_night_pickups})
    days.plot(kind='bar', stacked=True,figsize=(4.5, 4.5))
    plt.xlabel('Days of the week')
    plt.ylabel('Number of pickups')
    plt.tick_params(axis='both', which='minor')
    plt.plot()
    #plt.xticks(['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturd
    ay', 'Sunday'])
    days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturda
    y', 'Sunday']
    for i in range(0,len(days)):
        print ("{} - morning:{{}} , day:{{}} , mid-day:{{}} night:{{}}".format(day
        s[i],mid_night_pickups[i],day_pickups[i],afternoon_pickups[i],night_pickups
        [i]))

    return time

```

```
In [30]: times_of_day_dframe = compute_pickups_per_day()
```

```
Monday - morning:108576 , day:408179 , mid-day:434850 night:381266
Tuesday - morning:76339 , day:372232 , mid-day:415968 night:509383
Wednesday - morning:104280 , day:481818 , mid-day:488264 night:609625
Thursday - morning:260968 , day:545611 , mid-day:606775 night:754859
Friday - morning:200685 , day:547113 , mid-day:629661 night:849728
Saturday - morning:433710 , day:399576 , mid-day:724563 night:821859
Sunday - morning:377923 , day:276211 , mid-day:525948 night:403016
```



Inference:

1. More commute happens during the day[6:00 to 18:59 hrs] time compared to the night[19:00 to 23:59 hrs] or morning[00:00 to 5:59 hrs]
2. Saturday seems to be the busiest days for the taxi drivers overall
3. Most midnight commutes happen on Saturdays
4. Interestingly more commute happens during the midnight compared to on Sunday than on Saturday
5. Midnight commute during on Saturdays are the highest and consecutively Friday night commutes seem to be the highest(Friday night parties)

```

In [31]: #pickups and distances travelled at what time of the day??
def pickup_in_day(frame):
    hour_pickups = []
    temp = []
    for i in range(1,8):
        for j in range(0,24):
            temp.append(frame[(frame.pickup_day == i) & (frame.pickup_hour
== j)].shape[0])
        hour_pickups.append(temp)
        temp = []
    colors = ['xkcd:blue','xkcd:orange','xkcd:brown','xkcd:coral','xkcd:magenta',
'xkcd:green','xkcd:fuchsia']
    days = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday',
'y','Sunday']

    plt.figure(figsize=(8,4))
    hours_lis = [s for s in range(0,24)]
    for k in range(0,7):
        plt.plot(hours_lis,hour_pickups[k],colors[k],label = days[k])
        plt.plot(hours_lis,hour_pickups[k], 'ro', markersize=2)

    plt.xticks([s for s in range(0,24)])
    plt.xlabel('Hours of a day')
    plt.ylabel('Number of pickups')
    plt.title('Pickups for every hour')
    plt.legend()
    plt.grid(True)
    plt.show()

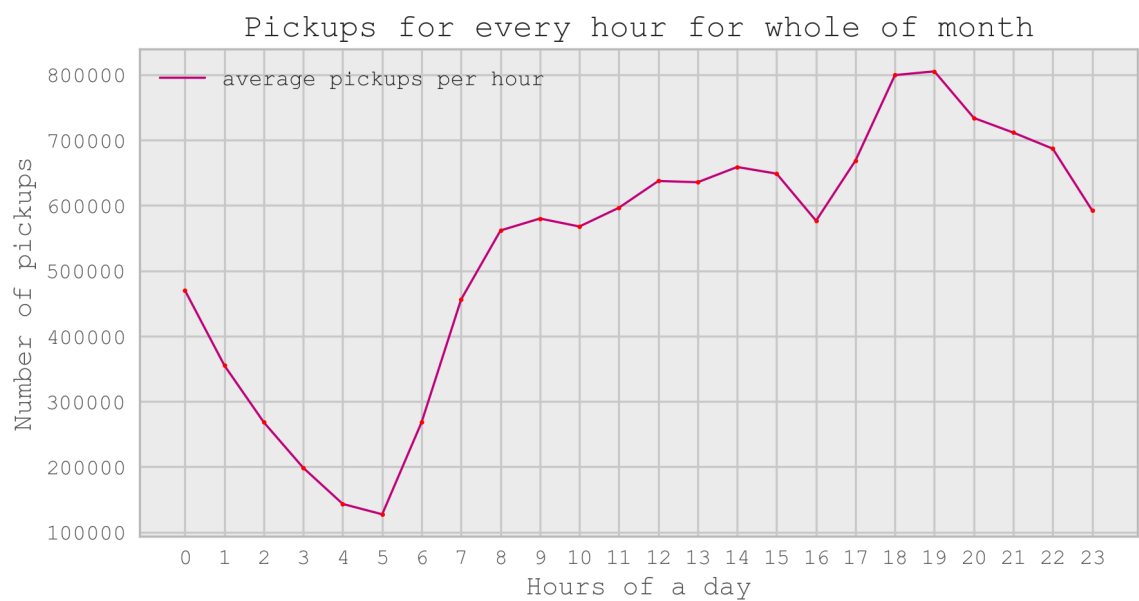
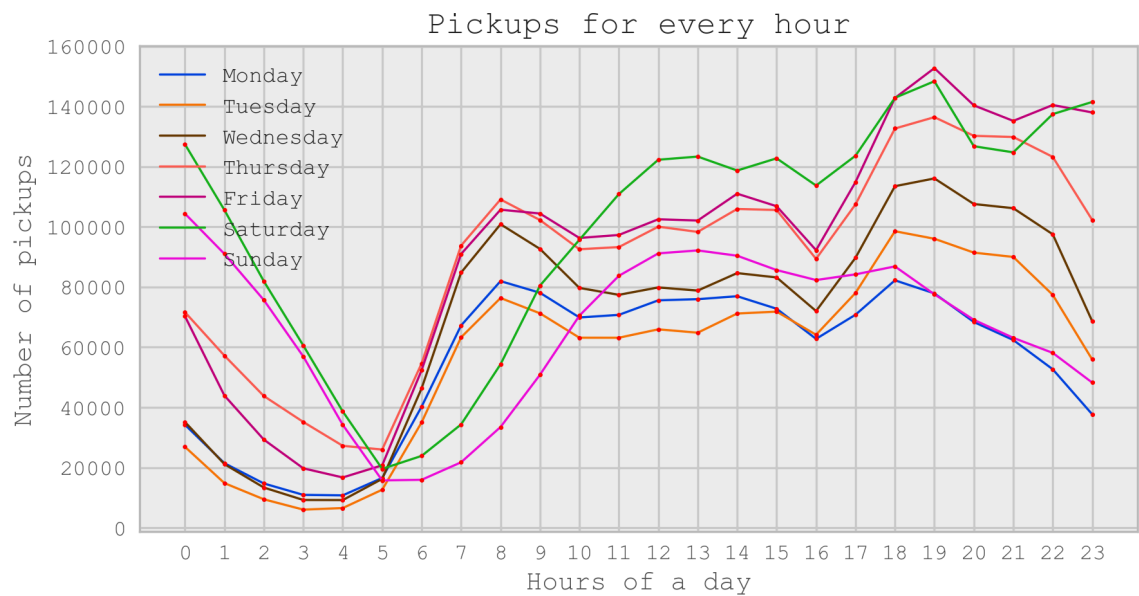
    hour_pickup_month = []
    for j in range(0,24):
        hour_pickup_month.append(frame[frame.pickup_hour == j].shape[0])

    plt.figure(figsize=(8,4))
    hours_lis = [s for s in range(0,24)]
    plt.plot(hours_lis,hour_pickup_month,'xkcd:magenta',label = 'average pi
ckups per hour')
    plt.plot(hours_lis,hour_pickup_month, 'ro', markersize=2)

    plt.xticks([s for s in range(0,24)])
    plt.xlabel('Hours of a day')
    plt.ylabel('Number of pickups')
    plt.title('Pickups for every hour for whole of month')
    plt.legend()
    plt.grid(True)
    plt.show()

pickup_in_day(times_of_day_dframe)

```



Clustering of regions:

```
In [32]: frame_with_durations_outliers_removed['lat_bin'] = np.round(frame_with_durations_outliers_removed['pickup_latitude'],4)
frame_with_durations_outliers_removed['lon_bin'] = np.round(frame_with_durations_outliers_removed['pickup_longitude'],4)
```

```
In [33]: #Find the best cluster where least number of clusters have a min. radius of
         #atleast 1km or more
         coords = frame_with_durations_outliers_removed[['pickup_latitude', 'pickup_
         longitude']].values

         def find_min_distance(cluster_centers, cluster_len):
             nice_points = 0
             wrong_points = 0
             for i in range(0, cluster_len-1):
                 min_value = 1000000
                 for j in range(i+1, cluster_len):
                     min_value = min(min_value, gpxpy.geo.haversine_distance(cluster_
                     _centers[i][0], cluster_centers[i][1], cluster_centers[j][0], cluster_center
                     s[j][1]))
                     if min_value!=1000000 and (min_value/1000) >= 1:
                         nice_points +=1
                     else:
                         wrong_points += 1
                 print (cluster_len, "Cents with min distance < 1:", wrong_points, "Cent
                 s with min distance > 1:", nice_points)

             if nice_points == cluster_len-1:
                 return 1
             else:
                 return 0

         def find_clusters(increment):
             kmeans = MiniBatchKMeans(n_clusters=increment, batch_size=10000).fit(co
             ords)
             frame_with_durations_outliers_removed['pickup_cluster'] = kmeans.predic
             t(frame_with_durations_outliers_removed[['pickup_latitude', 'pickup_longitu
             de']])
             cluster_centers = kmeans.cluster_centers_
             cluster_len = len(cluster_centers)
             return cluster_centers, cluster_len

         for increment in range(10, 100, 10):
             cluster_centers, cluster_len = find_clusters(increment)
             more_than_5km = find_min_distance(cluster_centers, cluster_len)
```

```
10 Cents with min distance < 1: 0 Cents with min distance > 1: 9
20 Cents with min distance < 1: 1 Cents with min distance > 1: 18
30 Cents with min distance < 1: 6 Cents with min distance > 1: 23
40 Cents with min distance < 1: 11 Cents with min distance > 1: 28
50 Cents with min distance < 1: 21 Cents with min distance > 1: 28
60 Cents with min distance < 1: 29 Cents with min distance > 1: 30
70 Cents with min distance < 1: 37 Cents with min distance > 1: 32
80 Cents with min distance < 1: 45 Cents with min distance > 1: 34
90 Cents with min distance < 1: 53 Cents with min distance > 1: 36
```

Inference:

- The main objective was to find a optimal min. distance(Which roughly estimates to the radius of a cluster) between the clusters which we got was 40

In [34]: *#Getting 40 clusters using the kmeans*

```
kmeans = MiniBatchKMeans(n_clusters=40, batch_size=10000).fit(coords)
frame_with_durations_outliers_removed['pickup_cluster'] = kmeans.predict(fr
ame_with_durations_outliers_removed[['pickup_latitude', 'pickup_longitud
e']])
```

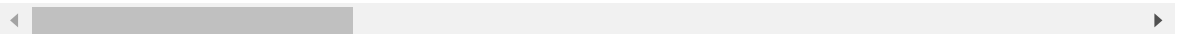
In [35]: *#Adding a new column describing what cluster the pickup point belongs to*

```
frame_with_durations_outliers_removed.head(5)
```

Out[35]:

	VendorID	payment_type	passenger_count	trip_distance	pickup_longitude	pickup
0	2	1	1	1.59	-73.993896	40.750
1	1	1	1	3.30	-74.001648	40.724
2	1	2	1	1.80	-73.963341	40.802
3	1	2	1	0.50	-74.009087	40.713
4	1	2	1	3.00	-73.971176	40.762

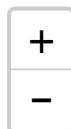
5 rows × 22 columns



In [36]: *#Plotting the cluster centers on OSM*

```
cluster_centers = kmeans.cluster_centers_
cluster_len = len(cluster_centers)
map_osm = folium.Map(location=[40.734695, -73.990372], tiles='Stamen Tone
r')
for i in range(cluster_len):
    folium.Marker(list((cluster_centers[i][0],cluster_centers[i][1])), popu
p=(str(cluster_centers[i][0])+str(cluster_centers[i][1]))).add_to(map_osm)
map_osm
```

Out[36]:



Leaflet (<http://leafletjs.com>) | Map tiles by Stamen Design (<http://stamen.com>), under CC BY 3.0 (<http://creativecommons.org/licenses/by/3.0>). Data by OpenStreetMap (<http://openstreetmap.org>), under ODbL (<http://www.openstreetmap.org/copyright>).

Plotting the clusters:

```
In [37]: def plot_clusters(frame):
    city_long_border = (-74.03, -73.75)
    city_lat_border = (40.63, 40.85)
    fig, ax = plt.subplots(ncols=1, nrows=1)
    ax.scatter(frame.pickup_longitude.values[:100000], frame.pickup_latitude.values[:100000], s=10, lw=0,
               c=frame.pickup_cluster.values[:100000], cmap='tab20', alpha=
0.2)
    ax.set_xlim(city_long_border)
    ax.set_ylim(city_lat_border)
    ax.set_xlabel('Longitude')
    ax.set_ylabel('Latitude')
    plt.show()

plot_clusters(frame_with_durations_outliers_removed)
```

